

RDT Mail Service documentation

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

src

src package

Subpackages

src.auth package

Submodules

src.auth.user module

Domain model representing an authenticated user and mailbox owner.

This module defines the *User* class, which encapsulates credential storage, mailbox path resolution, and serialization helpers used by authentication and mailbox subsystems.

```
class src.auth.user.User(username: str, password: str = '__internal__')
```

Bases: **object**

Represent an authenticated user and associated mailbox location.

Instances track normalized usernames, raw passwords used by the simulation, and the filesystem path to the user's mailbox root.

```
classmethod from_dict(data: Dict[str, Any]) → User
```

Reconstruct a *User* from a dictionary payload.

Parameters: **data** (*Dict[str, Any]*) – Mapping containing serialized user fields.

Returns: New instance created from the provided data.

Return type: [User](#)

Raises: **ValueError** – If required fields are missing or not valid strings.

```
set_password(raw_password: str) → None
```

Replace the stored password with a new value after validation.

Parameters: **raw_password** (*str*) – New password to associate with the user.

Raises: **ValueError** – If the provided password is empty or not a string.

```
to_dict() → Dict[str, str]
```

Serialize user state for JSON persistence.

Returns: Mapping containing normalized username and raw password.

Return type: Dict[str, str]

to_string() → str

Produce a diagnostic representation of the user state.

Returns: Multi-line summary that omits the raw password but reports mailbox path and metadata about credential presence.

Return type: str

verify_password(raw_password: str) → bool

Compare the provided password with the stored value.

Parameters: **raw_password** (str) – Candidate password to check.

Returns: True when the provided password matches the stored value; otherwise False.

Return type: bool

src.auth.user_manager module

User lifecycle management and authentication utilities.

The user manager is a singleton responsible for loading, persisting, and authenticating user accounts defined in the simulation. It coordinates access to the on-disk users.json store while providing thread-safe retrieval and creation of *User* objects.

`class src.auth.user_manager.UserManager`

Bases: **object**

Thread-safe singleton managing user persistence and authentication.

This manager encapsulates access to the JSON backing store and ensures that concurrent server threads can safely create users, authenticate credentials, and inspect cached user objects without corrupting state.

authenticate(username: str, password: str) → [User](#) | None

Validate credentials against the cached user records.

Parameters: • **username** (str) – Username attempting to authenticate.

• **password** (str) – Provided raw password.

Returns: User instance when authentication succeeds; otherwise None.

Return type: Optional[[User](#)]

create_user(username: str, password: str) → [User](#)

Create a new user, persist credentials, and initialize mailbox storage.

Parameters: • **username** (str) – Username for the new account.

• **password** (str) – Raw password to associate with the user.

Raises: **ValueError** – If the user already exists.

Returns: Newly created user object.

Return type: [User](#)

get_user(*username: str*) → [User](#) | None

Retrieve a cached user by username.

Parameters: **username** (*str*) – Username to look up; case-insensitive.

Returns: Matching user instance when present; otherwise None.

Return type: Optional[[User](#)]

to_string() → str

Returns a detailed diagnostic overview of the UserManager, including cache state, database file details, and locking info.

Module contents

src.common package

Submodules

src.common.exceptions module

File: src/common/exceptions.py

Description:

Custom exception classes for the Email Network Service. These allow the application to handle specific protocol failures gracefully (e.g., distinguishing between a network timeout and a bad password).

```
exception src.common.exceptions.EmailServiceError(message: str = '', *,
context: Mapping[str, Any] | None = None)
```

Bases: **Exception**

Base exception class for all custom errors in this project.

```
exception src.common.exceptions.POP3ConnectionError(message: str, *, host:
str | None = None, port: int | None = None, reason: str | None = None)
```

Bases: **EmailServiceError**

Raised when connection to the POP3 server fails.

```
exception src.common.exceptions.POP3ProtocolError(message: str, *, command:
str | None = None, response: str | None = None)
```

Bases: **EmailServiceError**

Raised when the POP3 conversation violates protocol rules. .. rubric:: Examples

- Authentication failure (-ERR response).
- Malformed server responses.

```
exception src.common.exceptions.RDTPacketError(message: str, *,
packet_bytes: bytes | None = None, detail: str | None = None)
```

Bases: **EmailServiceError**

Raised when a packet is corrupt or cannot be parsed.

```
exception src.common.exceptions.RDTTimeoutError(message: str, *, seq: int |
None = None, retries: int | None = None, timeout: float | None = None)
```

Bases: **EmailServiceError**

Raised when the RDT layer times out waiting for an expected packet.

```
exception src.common.exceptions.SMTPConnectionError(message: str, *, host:
str | None = None, port: int | None = None, reason: str | None = None)
```

Bases: **EmailServiceError**

Raised when the connection to the SMTP server fails. .. rubric:: Examples

- Server is down / unreachable.
- Connection timed out during handshake.
- RDT transport layer failure.

```
exception src.common.exceptions.SMTPProtocolError(message: str, *, code: int
| None = None, command: str | None = None, response: str | None = None)
```

Bases: **EmailServiceError**

Raised when the SMTP conversation violates the protocol rules. .. rubric:: Examples

- Receiving a 500 error from the server.
- Receiving garbage data instead of a response code.
- Sending commands out of order.

src.common.logger module

```
class src.common.logger.ColoredFormatter(fmt=None, datefmt=None, style='%',
validate=True, *, defaults=None)
```

Bases: **Formatter**

Custom formatter that adds ANSI colors to the levelname. Ensures alignment by padding the level name before applying color codes.

```
COLORS = {10: '\x1b[36m', 20: '\x1b[32m', 30: '\x1b[33m', 40: '\x1b[31m',
50: '\x1b[35m'}
```

```
RESET = '\x1b[0m'
```

```
format(record)
```

Format the specified record as text.

The record's attribute dictionary is used as the operand to a string formatting operation which yields the returned string. Before formatting the dictionary, a couple of preparatory steps are carried out. The message attribute of the record is computed using `LogRecord.getMessage()`. If the formatting string uses the time (as determined by a call to `usesTime()`, `formatTime()` is called to format the event time. If there is exception information, it is formatted using `formatException()` and appended to the message.

```
src.common.logger.get_logger(key: str) → Logger
```

```
src.common.logger.log_block(key: str, title: str, origin: str | None = None)
```

```
src.common.logger.log_block_detailed(title: str, origin: str | None = None)
```

Block logger for the detailed log file.

```
src.common.logger.log_block_simple(title: str, origin: str | None = None)
```

Block logger for the simple log file.

```
src.common.logger.log_debug(key: str, msg: str, origin: str | None = None)
```

```
src.common.logger.log_debug_detailed(msg: str, origin: str | None = None, **kwargs)
```

```
src.common.logger.log_error(key: str, msg: str, origin: str | None = None)
```

```
src.common.logger.log_error_detailed(msg: str, origin: str | None = None, **kwargs)
```

```
src.common.logger.log_info(key: str, msg: str, origin: str | None = None)
```

```
src.common.logger.log_info_detailed(msg: str, origin: str | None = None, **kwargs)
```

```
src.common.logger.log_warning(key: str, msg: str, origin: str | None = None)
```

```
src.common.logger.log_warning_detailed(msg: str, origin: str | None = None, **kwargs)
```

```
src.common.logger.register_logger(key: str, name: str, level: str = 'INFO', log_file: str | None = None) → Logger
```

Creates a named logger pipeline that writes to both Console and File.

Module contents

src.mailbox package

Submodules

src.mailbox.mailbox_reader module

Read-only accessors for mailbox contents and metadata.

The mailbox reader loads metadata, enumerates stored messages, retrieves individual files, and exposes UID lookup helpers required by POP3 commands. All operations are non-destructive and intended for concurrent read access.

`class src.mailbox.mailbox_reader.MailboxReader`

Bases: **object**

Provide read-only operations over a user's mailbox directory.

The reader consumes metadata.json to support listing and UIDL semantics and exposes helpers for loading message bodies for POP3 RETR operations.

get_uid(*user*: [User](#), *filename*: *str*) → *str* | *None*

Retrieve the UID recorded for a specific message.

Parameters:

- **user** ([User](#)) – Owner of the mailbox.
- **filename** (*str*) – Identifier of the message whose UID is requested.

Returns: UID value from metadata when present; otherwise *None*.

Return type: Optional[*str*]

list_messages(*user*: [User](#)) → List[Tuple[*str*, *int*, *str*]]

List visible messages for the given user.

The listing excludes entries marked as deleted and provides filename, size, and UID tuples suitable for POP3 LIST, STAT, and UIDL responses.

Parameters: **user** ([User](#)) – Account whose mailbox is being inspected.

Returns: Tuples of (*filename*, *size_bytes*, *uid*) for each non-deleted message.

Return type: List[Tuple[*str*, *int*, *str*]]

read_message(*user*: [User](#), *filename*: *str*) → *str* | *None*

Load the raw contents of a stored message.

Parameters:

- **user** ([User](#)) – Owner of the mailbox.
- **filename** (*str*) – Message filename to retrieve.

Returns: Message text when the file exists and can be read; otherwise *None*.

Return type: Optional[*str*]

to_string() → *str*

Render a formatted diagnostic of reader configuration.

Returns: Summary of mailbox paths used for read access.

Return type: *str*

src.mailbox.mailbox_writer module

Facilities for writing messages and metadata into mailbox storage.

The mailbox writer manages atomic writes of MIME messages to per-user mailboxes and maintains the accompanying metadata.json entries that track sizes, deletion flags, and stable unique identifiers.

`class src.mailbox.mailbox_writer.MailboxWriter`

Bases: **object**

Handle persistent writes of messages and metadata to user mailboxes.

The writer ensures that messages are staged to a temporary location before atomic replacement to guard against partial writes, and it updates mailbox metadata with size and UID information expected by POP3 semantics.

mark_message_as_deleted(*user*: [User](#), *filename*: *str*) → bool

Mark a message entry as deleted without removing the file.

Parameters:

- **user** ([User](#)) – Owner of the mailbox.
- **filename** (*str*) – Identifier of the message to flag.

Returns: True when the metadata flag was updated; False if the message entry does not exist.

Return type: bool

to_string() → *str*

Render a formatted representation of writer configuration.

Returns: Summary of mailbox and temporary storage roots.

Return type: *str*

write_email(*user*: [User](#), *email_data*: *EmailData*) → *str* | None

Persist a MIME message to the user's mailbox and record metadata.

The method stages the message in a temporary directory, atomically moves it into the mailbox, and updates metadata with UID and size information.

Parameters:

- **user** ([User](#)) – Recipient account for the message.
- **email_data** (*EmailData*) – Email payload, including MIME representation and optional UID.

Returns: Path to the finalized message file on success; None when the write or metadata update fails.

Return type: Optional[*str*]

src.mailbox.storage_manager module

Thread-safe orchestration layer for mailbox persistence and retrieval.

The storage manager coordinates mailbox operations across SMTP and POP3 handlers while enforcing per-user locking. It maintains a singleton instance that hands off work to reader and writer helpers and guarantees that mailbox directories and metadata files remain consistent under concurrent access.

```
class src.mailbox.storage_manager.StorageManager
```

Bases: **object**

Centralized, thread-safe controller for mailbox persistence.

The storage manager is implemented as a singleton to ensure that a single coordinator enforces locking semantics for all mailbox interactions. It proxies read and write operations to dedicated helper classes while protecting per-user resources with granular locks so that concurrent SMTP and POP3 operations cannot corrupt metadata or message files.

```
delete_email(user: User, filename: str) → bool
```

Mark a message as deleted within metadata.

The method does not remove the underlying file. It sets the deleted flag under a per-user lock so POP3 will exclude the message from listings.

Parameters:

- **user** ([User](#)) – The mailbox owner.
- **filename** (*str*) – The message identifier to mark.

Returns: True if the deletion flag was recorded; otherwise False.

Return type: bool

```
get_lock(username: str) → allocate_lock
```

Retrieve or lazily create the mutex protecting a user's mailbox.

Parameters: **username** (*str*) – The username whose mailbox operations must be synchronized.

Returns: The lock associated with the provided username.

Return type: threading.Lock

```
get_message_content(user: User, filename: str) → str | None
```

Retrieve the raw contents of a stored message.

The method acquires the user-level lock before delegating the read to the mailbox reader. Missing files or I/O failures are logged and surfaced as a *None* return value.

Parameters:

- **user** ([User](#)) – The mailbox owner.
- **filename** (*str*) – The message filename to load.

Returns: Full message text when available; otherwise *None*.

Return type: Optional[str]

```
get_message_uid(user: User, filename: str) → str | None
```

Look up the stable UID associated with a message file.

Parameters:

- **user** ([User](#)) – The mailbox owner.
- **filename** (*str*) – The message filename whose UID is requested.

Returns: The UID string if present in metadata; otherwise *None*.

Return type: Optional[str]

```
list_messages(user: User) → List[Tuple[str, int, str]]
```


Enumerate non-deleted messages for a user.

The method returns the filename, size, and UID for each retained message by reading metadata under a per-user lock.

Parameters: **user** ([User](#)) – The account whose mailbox should be listed.

Returns: Sequence of *(filename, size_bytes, uid)* tuples for each visible message. Deleted messages are excluded.

Return type: List[Tuple[str, int, str]]

save_email(*user: User, email_data: EmailData*) → str | None

Persist an email atomically for the provided user.

The method acquires the per-user lock, delegates disk I/O to the *MailboxWriter*, and releases the lock regardless of success. Errors are logged and surfaced as a *None* return value.

Parameters: • **user** ([User](#)) – The mailbox owner receiving the message.

• **email_data** (*EmailData*) – Structured representation of the message to store.

Returns: Absolute path to the finalized message file when persistence succeeds; otherwise *None*.

Return type: Optional[str]

to_string() → str

Produce a formatted diagnostic snapshot of manager state.

Returns: Human-readable summary of collaborators and lock statistics.

Return type: str

Module contents

src.pop3 package

Submodules

src.pop3.pop3_client module

`class src.pop3.pop3_client.POP3Client`

Bases: **object**

authenticate(*user: str, password: str*)

connect()

delete(*msg_num: int*)

get_all_messages(*username, password*) → List[str]

Connects, authenticates, retrieves all emails, deletes them, and quits. Returns a list of raw email strings.

list() → List[Tuple[int, int]]

quit()

retr(*msg_num: int*) → str

stat() → Tuple[int, int]

to_string() → str

Returns a diagnostic summary of the POP3Client state.

src.pop3.pop3_server module

class src.pop3.pop3_server.**POP3Server**(*host: str, port: int*)

Bases: **object**

start()

stop()

to_string() → str

Returns a pretty-formatted overview of the POP3Server's configuration and runtime state. Useful for debugging, diagnostics, and health checks.

Module contents

src.rdt package

Submodules

src.rdt.rdt_dispatcher module

class src.rdt.rdt_dispatcher.**RDTDischer**(*sock: socket*)

Bases: **object**

The Central Nervous System for the RDT protocol.

It owns the UDP socket and runs a background thread to continuously receive packets. It inspects every packet and routes it: 1. ACK Packets -> Routed to the specific RDTSender waiting for them. 2. Data Packets -> Pushed to a Queue for the RDTRceiver to consume.

get_data_packet(*timeout=None*) → Tuple[Dict, Tuple[str, int]] | None

Blocking call to get the next valid data packet.

register_ack_waiter(*host: str, port: int, seq: int*) → Event

Called by Sender BEFORE sending data.

start()

Starts the background listening thread.

stop()

Stops the background thread.

to_string() → str

Returns a detailed snapshot of dispatcher runtime state.

unregister_ack_waiter(*host: str, port: int, seq: int*)

Called by Sender after receiving ACK or timeout.

src.rdt.rdt_packet module

src.rdt.rdt_packet.**calculate_checksum**(*data: bytes*) → intComputes the CRC32 checksum for the given data. Ensures the result is an unsigned 32-bit integer (0 to 2³²-1).src.rdt.rdt_packet.**make_ack_packet**(*seq_num: int*) → bytessrc.rdt.rdt_packet.**make_data_packet**(*seq_num: int, data: bytes*) → bytessrc.rdt.rdt_packet.**make_packet**(*seq_num: int, is_ack: bool, payload: bytes*) → bytes

Constructs a binary packet. Format: [SEQ: 1B] [TYPE: 1B] [CHECKSUM: 4B] [PAYLOAD: N Bytes]

src.rdt.rdt_packet.**unpack_and_validate**(*packet_bytes: bytes*) → dict | None

Parses binary packet, verifies checksum, and returns a dict compatible with the rest of the application.

src.rdt.rdt_receiver module

class src.rdt.rdt_receiver.**RDTRceiver**(*dispatcher: [RDTRDispatcher](#)*)Bases: **object**

RDT 3.0 Receiver. Standardized to ALWAYS yield (data, sender_address).

reset_state(*addr: Tuple[str, int]*)

Resets RDT state for a client (used on QUIT).

start_receiving()

Generator that yields (valid_data_bytes, sender_addr).

stop()**to_string()** → str

Returns the runtime state of the RDTRceiver.

src.rdt.rdt_sender module

class src.rdt.rdt_sender.**RDTRSender**(*dest_host: str, dest_port: int, dispatcher: [RDTRDispatcher](#)*)Bases: **object**

RDT 3.0 Sender. Refactored to use `RDTDDispatcher` for ACK handling to prevent socket monopolization.

close()

send(*data_chunk: bytes*)

Reliable send. Blocks until ACK is received or max retries exhausted.

to_string() → `str`

Returns a diagnostic summary of the sender's connection state.

Module contents

src.smtp package

Submodules

src.smtp.smtp_client module

Implements an SMTP client for sending emails.

This module provides *SMTPClient*, a class that handles the client-side of the SMTP protocol. It uses the Reliable Data Transfer (RDT) protocol over UDP to communicate with an SMTP server. The client is responsible for the entire email sending transaction, from the initial handshake to sending the email data and closing the connection.

class src.smtp.smtp_client.SMTPClient

Bases: **object**

An SMTP client that sends emails using the RDT protocol over UDP.

This class encapsulates the entire SMTP workflow for sending an email. It manages a UDP socket, an RDT sender and receiver, and the sequence of SMTP commands required to complete an email transaction.

The client binds to an ephemeral port, allowing multiple instances to run concurrently.

log

A logger instance for the class.

sock

The UDP socket used for communication.

Type: `socket.socket`

dispatcher

The dispatcher managing incoming RDT packets.

Type: [RDTDDispatcher](#)

rdt_sender

The RDT sender for reliable data transmission.

Type: [RDTSender](#)

rdt_receiver

The RDT receiver for reliable data reception.

Type: [RDTRceiver](#)

receiver_gen

A generator for receiving RDT packets.

is_connected

A flag indicating the connection state.

Type: bool

send_email(*sender: str, recipient: str, subject: str, body: str*) → bool

Sends an email by executing the full SMTP transaction.

Parameters:

- **sender** (*str*) – The sender’s email address.
- **recipient** (*str*) – The recipient’s email address.
- **subject** (*str*) – The subject of the email.
- **body** (*str*) – The body content of the email.

Returns: True if the email was sent successfully, False otherwise.

Return type: bool

to_string() → str

Returns a detailed diagnostic overview of the SMTPClient state, including socket info, dispatcher status, RDT components, and connection flags.

src.smtp.smtp_server module

```
class src.smtp.smtp_server.SMTPServer(host: str, port: int)
```

Bases: **object**

start()

stop()

to_string() → str

Returns a pretty-formatted overview of the POP3Server’s configuration and runtime state. Useful for debugging, diagnostics, and health checks.

```
class src.smtp.smtp_server.SMTPState(value)
```

Bases: **Enum**

READING_DATA_STREAM = 5

```
WAIT_FOR_DATA_CMD = 4
WAIT_FOR_HELO = 1
WAIT_FOR_MAIL_FROM = 2
WAIT_FOR_RCPT_TO = 3
```

Module contents

Submodules

src.config module

```
class src.config.Config
```

Bases: **object**

Central OOP-based configuration object for the email system. All settings are class-level constants and may be overridden at runtime via CLI arguments (see `apply_args()`).

```
AUTOLOGIN_FILE = '.autologin.json'
```

```
BASE_DIR =
'/Users/andrewphotinakis/Documents/Fall2025/CSCI651/GroupProject/smtp-protocol-implementation/email-network-service'
```

```
CLIENT_IP = '127.0.0.1'
```

```
CLIENT_LISTENING_PORT = 2526
```

```
CONCURRENCY = 5
```

```
DATABASE_PATH =
'/Users/andrewphotinakis/Documents/Fall2025/CSCI651/GroupProject/smtp-protocol-implementation/email-network-service/database'
```

```
DELAY_BETWEEN_SENDS = 0.1
```

```
LOGS_DIR =
'/Users/andrewphotinakis/Documents/Fall2025/CSCI651/GroupProject/smtp-protocol-implementation/email-network-service/logs'
```

```
LOG_LEVEL = 'DEBUG'
```

```
MAILBOXES_DIR =
'/Users/andrewphotinakis/Documents/Fall2025/CSCI651/GroupProject/smtp-protocol-implementation/email-network-service/database/mailboxes'
```

```
MAX_PAYLOAD_SIZE = 1024
```

```
MESSAGE_SIZE = 256
```

```
NUM_EMAILS_PER_USER = 10
```

```
NUM_USERS = 10
```

```

POP3_SERVER_HOST = '127.0.0.1'

POP3_SERVER_PORT = 1100

RDT_RECV_BUFSIZE = 4096

RDT_TIMEOUT = 1.0

RDT_WINDOW_SIZE = 10

SERVER_BIND_IP = '127.0.0.1'

SMTP_SERVER_HOST = '127.0.0.1'

SMTP_SERVER_PORT = 2525

TEMP_EMAILS_DIR =
'/Users/andrewphotinakis/Documents/Fall2025/CSCI651/GroupProject/smtp-protocol-implementation/email-network-service/database/temp'

USER_DB_FILE =
'/Users/andrewphotinakis/Documents/Fall2025/CSCI651/GroupProject/smtp-protocol-implementation/email-network-service/database/users.json'

classmethod apply_args(args)

classmethod create_required_dirs()
    Ensure essential directories exist.

classmethod log_config()
    Log all configuration values.

```

src.simulation module

ADVANCED NETWORKED EMAIL SYSTEM SIMULATION

```

class src.simulation.Simulation(num_users, num_emails, concurrency,
delay_between_sends, message_size)
    Bases: object

    ensure_directories()
        Clean and recreate mailbox + temp directories.

    ensure_users_json()
        Reset users.json to { 'users': [] }.

    run()

    run_email_simulation()

    run_pop3_integrity_check()
        Run POP3 verification in parallel using ThreadPoolExecutor. Each user's mailbox is checked
        concurrently.

```

setup_directories()**setup_users()****start_servers()****stop_servers()****to_string()** → str

Returns a detailed diagnostic overview of the Simulation environment, configuration, internal state, and runtime components.

class src.simulation.SimulationStatsBases: **object**

Thread-safe statistics aggregator.

print_summary()**record_pop3**(*verified: bool*)**record_smtp**(*success: bool, payload_size: int*)**start_timer()****stop_timer()****src.simulation.main()****src.simulation.parse_args()**

Module contents